D424 – Software Engineering

Task 3

Capstone Proposal Project Name: Trip Itinerary

Student Name: Zoey Hankins

Table of Contents

Table of Contents *2*

Application Design and Testing *4*

Class Design **4**

UI Design **5**

Unit Test Plan *7*

Introduction to Test Plan **7**

Purpose 7

Overview 7

Test Plan **8**

Items 8

Features 8

Deliverables 8

Tasks **8**

Needs 9

Pass/Fail Criteria 9

Specifications 10

Procedures **11**

Results **12**

Guide for Setting Up and Maintaining the App *13*

System Requirements **13**

Code Structure **13**

Database Structure **13**

Setting up the environment **13**

Writing Tests **14**

Building APK File **14**

User Guide ***15***

Introduction to User Guide **15**

Installation and Using the Application **15**

Installation Method 1: Using APK on an Android Device. 15

Installation Method 2: Using the APK file on PC through Android Studio. 19

Using the Application **24**

Login 24

Adding Trips 26

Adding Excursions 28

View and Search Vacation *Report* 30

Sharing Vacation Details 31

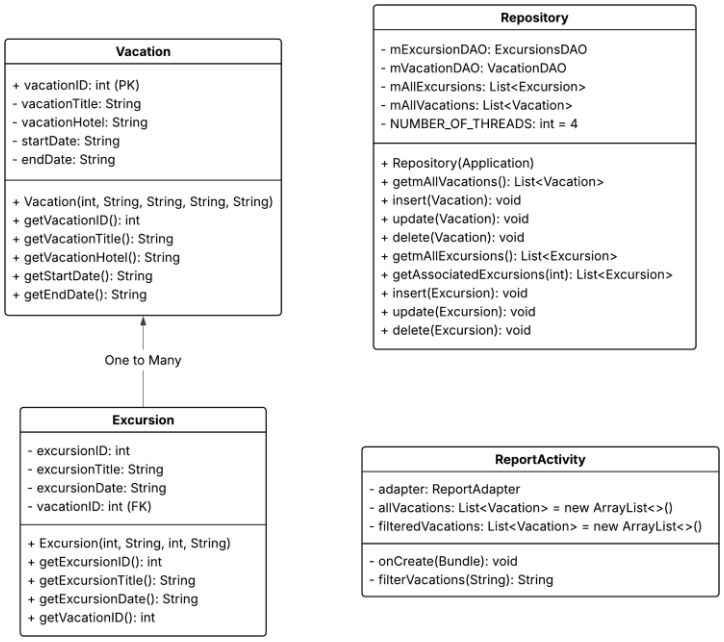
Deleting a Vacation 32

Application Design and Testing

Class Design

This UML class diagram accurately shows the main relationships between classes. This diagram, created with Lucidchart, illustrates how the classes interact while maintaining scalability. The four main classes that make up this Android application are ‘Vacation’, ‘Excursion’, ‘Repository’, and ‘ReportActivity’.

The ‘Vacation’ class represents a trip with attributes such as id, title, hotel name, start date, and end date. The ‘Vacation’ class has a one-to-many relationship with the ‘Excursion’ class, which stores all data for excursions associated with a vacation. The ‘Repository’ class handles data operations, such as adding, editing, deleting, and retrieving entries. The ‘ReportActivity’ class handles the ‘report’ page, where filtering and timestamp generation happen.



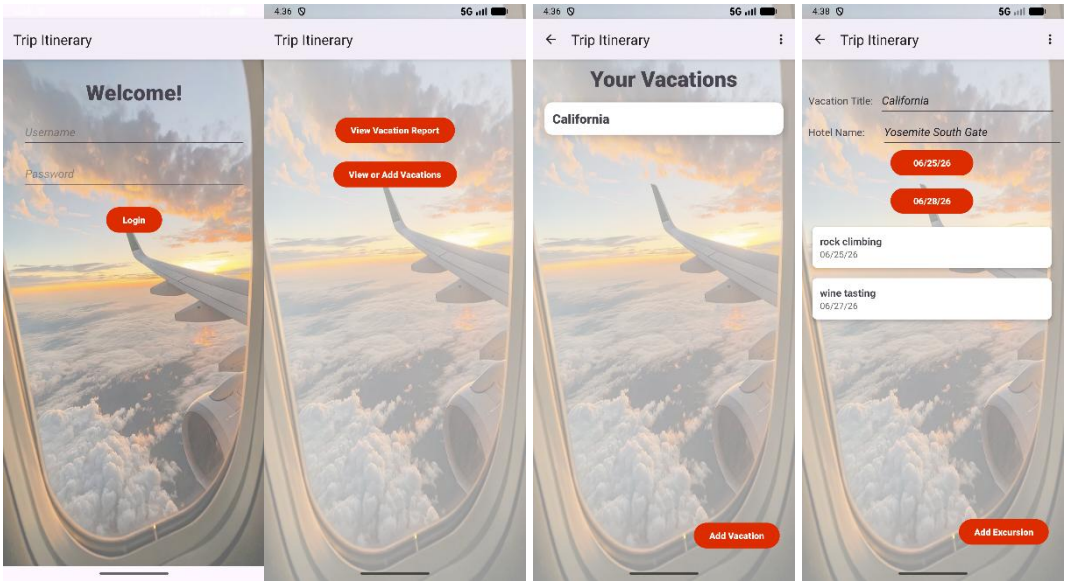
UI Design

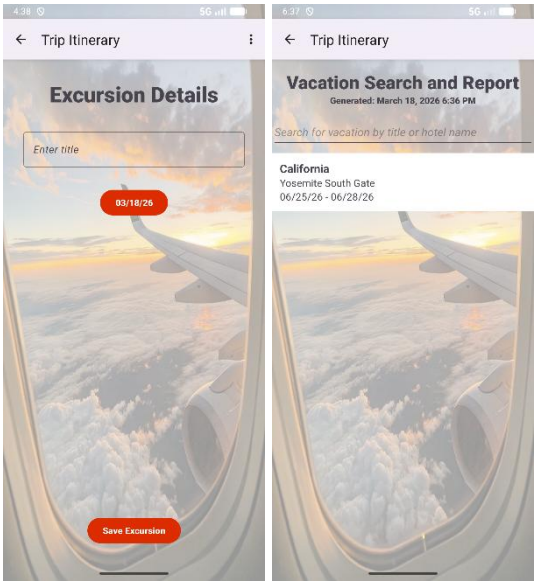
The Trip Itinerary starts with a simple login page. The user has a predefined username and a hashed predefined password. Once the user has logged in, there are two option buttons: “view the vacation report” and “view and add vacations”.

I added a vacation to show how the UI will look when there are vacations added. When the user chooses to view or add vacations, they are taken to a page showing all their vacations. They can click any vacation to edit or delete it, or to add, edit, or delete excursions associated with it. The user can use the dropdown menu in the top right to save, delete, or share a vacation.

When a user is ready to view the vacation report, they can click the top-left arrow to return to the previous page if they are on the vacation list page. Once the user clicks on the vacation report, all vacations are shown in more detail. The user can then search for the name of the vacation or the hotel, and all vacations that match that criteria will be shown to the user.

TRIP ITINERARY





Unit Test Plan

Introduction to Test Plan

Purpose

The testing will focus on the functionality of the login process. I set up testing to verify that user input is properly validated and that it directs the user to the correct screen or informs the user that the credentials are invalid if the user does not enter a password or does not capitalize the username correctly. Testing kept failing because of a mistake I made with the onView(withText()), but I resolved the bug by reading a Stack Overflow post. After fixing my error, testing passed.

Overview

Three tests are performed on the login page input to verify secure access and input validation.

1. Not entering any user credentials: The first test performed was to verify that if the user did not enter any credentials, the application would properly display "Invalid credentials" and not let the user go any further.
2. Entering the username while not capitalizing: The second test performed was to verify that even if the user entered the right predefined password, they would be shown "Invalid credentials" and not allowed in the application unless the predefined username is correct and includes its capital letters ("Trip User", not "trip user").
3. Entering the proper username and password: This is the third and last test performed. This is to verify that when the user enters the correct username and password, they can proceed using the application.

These tests are important for keeping the user information safe and secure, as the application promised. Even though in testing I did not pull any methods from the 'LoginActivity.java' file directly, the testing was inspired by a certain private function in that file. There is an 'attemptLogin' function that is what I reproduced in testing. The function takes the user's responses or non-responses and either prevents the user from proceeding or grants them access to the application.

The only roadblock I encountered was entirely user error, since this was my first time using Espresso. While using the onView(withText("")) Espresso code, I was unaware that within "", you must specify EXACTLY what you want the test to see on the user interface to verify success, down to capitalization. Once I figured that out and rewrote those small code sections, testing was successful.

Test Plan

Items

Source code, Android Studio & Emulator running API 36, JUnit4 framework, Espresso for UI testing, and AndroidX testing libraries.

Features

Activity navigation to 'MainActivity' page, 'LoginActivity' login form functionality.

Deliverables

Screenshot of test code from 'LoginActivityTest' and screenshot of test results.

Tasks

The first task was to create a 'LoginActivityTest' file. The second task was to pick which libraries/frameworks to use. I then began writing code according to the JUnit4 and Espresso user guides, and Android Studio started prompting me to import these libraries. After writing code

according to my testing plan, I run the tests, identify any issues, rewrite code as needed, and record and take screenshots of the passing test results.

Needs

Android Studio is used for testing with an emulator running API 36, but the emulator must be initialized for the tests to start. The Espresso and JUnit4 frameworks were implemented for this specific testing, and predefined credentials were required to input for testing.

Pass/Fail Criteria

For a test to be considered passing, it must be based on two parameters. The user and password were either nonexistent or incorrect, resulting in a “Invalid credentials” message. When “Invalid credentials” is shown, the test has passed. The other passing test is that when the correct credentials are used, the application navigates to the ‘MainActivity’ screen and displays a “View Vacation Report” button.

Specifications

This is the code that was written for testing.

LoginActivityTest.java × LoginActivity.java

```java
package com.example.tripitinerary;

import static androidx.test.espresso.Espresso.*;
import static androidx.test.espresso.action.ViewActions.*;
import static androidx.test.espresso.assertion.ViewAssertions.*;
import static androidx.test.espresso.matcher.ViewMatchers.*;

import androidx.test.ext.junit.rules.ActivityScenarioRule;
import androidx.test.ext.junit.runners.AndroidJUnit4;

import com.example.tripitinerary.UI.LoginActivity;

import org.junit.Rule;
import org.junit.Test;
import org.junit.runner.RunWith;

@RunWith(AndroidJUnit4.class)
public class LoginActivityTest {
    @Rule
    public ActivityScenarioRule<LoginActivity> activityRule = new ActivityScenarioRule<>(LoginActivity.class);

    @Test
    public void testEmptyFields() {
        onView(withId(R.id.loginButton)).perform(click());
        onView(withText("Invalid credentials")).check(matches(isDisplayed()));
    }

    @Test
    public void testWithFieldsNotCapitalized() {
        onView(withId(R.id.usernameField)).perform(typeText( stringToBeTyped: "trip user"));
        onView(withId(R.id.passwordField)).perform(typeText( stringToBeTyped: "password"));
        onView(withId(R.id.loginButton)).perform(click());
        onView(withText("Invalid credentials")).check(matches(isDisplayed()));
    }

    @Test
    public void testWithFields() {
        onView(withId(R.id.usernameField)).perform(typeText( stringToBeTyped: "Trip User"));
        onView(withId(R.id.passwordField)).perform(typeText( stringToBeTyped: "password"));
        onView(withId(R.id.loginButton)).perform(click());
        onView(withText("View Vacation Report")).check(matches(isDisplayed()));
    }
```

Procedures

My first step to test was to research which frameworks I wanted to use to write the code. I found that JUnit4 is easy to use with Android Studio testing, and that Espresso provides the methods I needed to yield the results I wanted, such as View Actions and View Matchers, and they both work well with my Gradle build. Once I found the frameworks I wanted, I used the user guides to add Espresso and JUnit4 dependencies to my Gradle build and set my instrumentation runner to the config file.

My next step was to create the test file. Once the file was created and the main method was defined, I started writing code using the framework guides. While I was writing code, Android Studio would detect a method from a specific library and prompt me to import it, which reduced the research load. I wrote the test code using the user guides and, when I was done, let Android Studio import all the libraries used at the top of the file.

Once I finished writing the code, I tried to run the tests using the Android Studio Emulator with API 36, but they all failed. I kept researching the error in the console log, and couldn't figure it out until I found a Stack Overflow post with the answer I needed. The problem was that I wrote

```
onView(withText("Invalid Credentials")).check(matches(isDisplayed()));
```

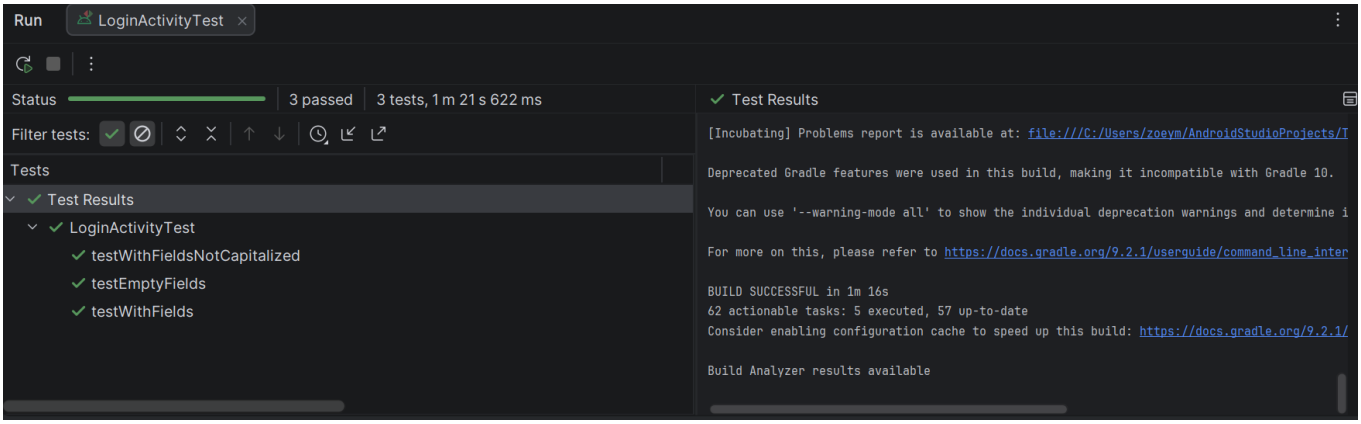
instead of

```
onView(withText("Invalid credentials")).check(matches(isDisplayed()));
```

which made the test fail because the emulator isn't displaying the UI text EXACTLY as it is written in the source code. A simple mistake, but thankfully solved. Once that was resolved, all three tests ran and passed.

Results

These are the results the test yielded.



.

Guide for Setting Up and Maintaining the App

https://zoeyhankins.github.io/Trip_Itinerary_Website/

System Requirements

1. **Operating System:** Windows 10/11, macOS 10+, or Linux Ubuntu 18+
2. **Android Studio:** Panda 2 2025.3.2
3. **SDK:** Android 36.0 Baklava

Code Structure

\app\src\main\java\com\example\tripitinerary\

1. DAO – DAO Implementation for Excursion and Vacation
2. Database – Database for Repository and Vacation Database Builder
3. Entities – Entities for Excursion and Vacation
4. UI – All UI activity

Database Structure

1. Repository: holds all other CRUD functionality and Vacation and Excursion details.
2. VacationDB: Utility class to handle database operations

Setting up the environment

1. Download the .ZIP file from the link above.
2. Extract the .ZIP file.
3. Open the folder in Android Studio.
4. Let Gradle sync.
5. Build project. “Assemble ‘app’ Run configuration.

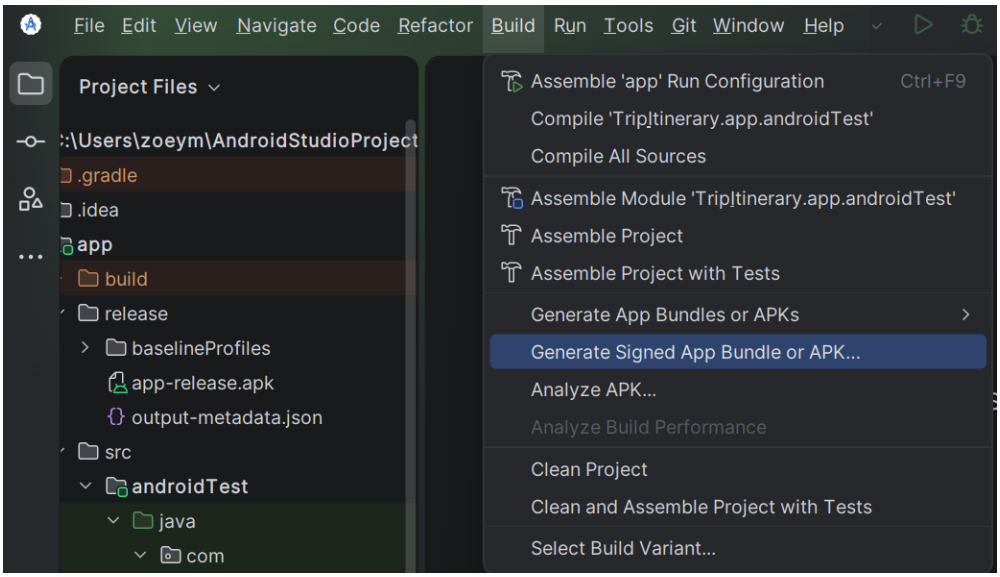
Writing Tests

app/src/androidTest/java/com/example/tripitinerary/

1. Here you can see the LoginActivityTest that was made.
2. Create your test files here.

Building APK File

1. Go to the settings at the top left.
2. Find the “Build” tab.
3. Click “Generate Signed App Bundle or APK”.
4. Click APK.
5. Fill out keystore information.
6. Sign the APK as a release APK, not a debug APK file.



User Guide

Introduction to User Guide

This guide gives clear instructions for installing and using the Trip Itinerary Android Application. This guide walks you through installing the application via APK file, logging in, viewing, and managing trips and excursions, and generating your trip report. All steps in this guide have been tested to ensure simplicity for all users.

Installation and Using the Application

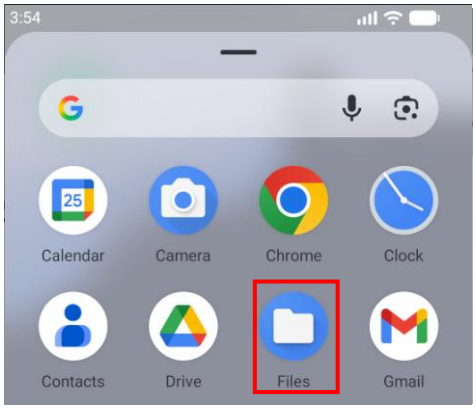
Installation Method 1: Using APK on an Android Device.

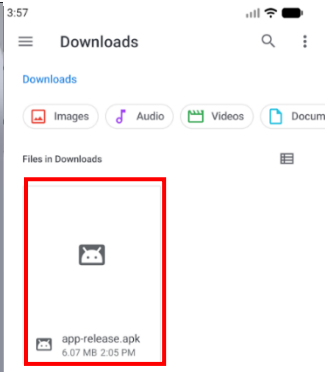
1. Download the APK file from the webpage.

   https://zoeyhankins.github.io/Trip_Itinerary_Website/

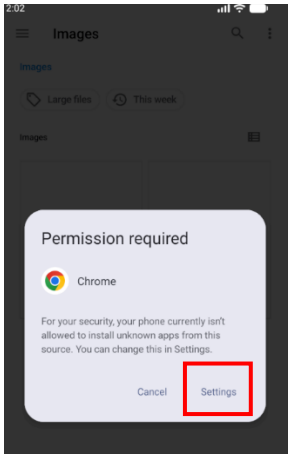
   The APK file will end up in a Downloads folder, or somewhere else if you specified.

2. When the APK file is done downloading, you will go to the app drawer and open your files application. The APK should be downloaded here.



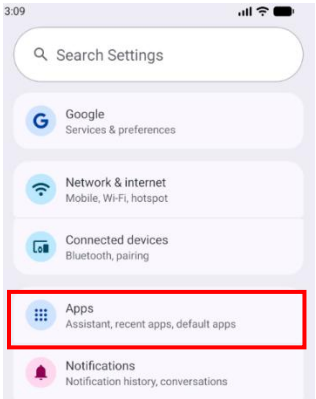


3. When you find the APK file, click on it to open it. This prompt should pop up, and you will click the settings button. If it doesn't pop up, open your settings.

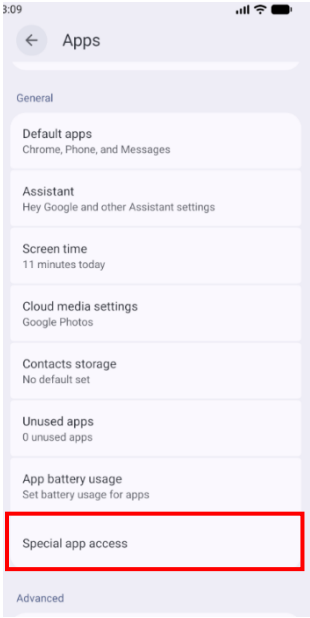


4. **If the "Permission required" prompt popped up for you, skip to step 8.**

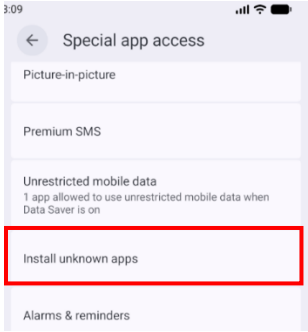
   If the prompt didn't pop up for you, while in settings, go to "Apps" and click on it.



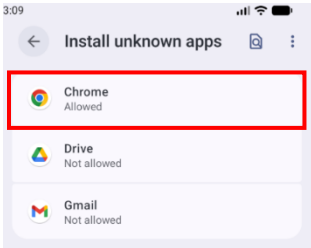
5. Then scroll down to find “Special App Access” and click that.



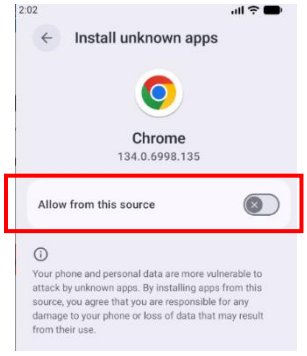
6. Then scroll down to find “Install unknown apps”, click on that.



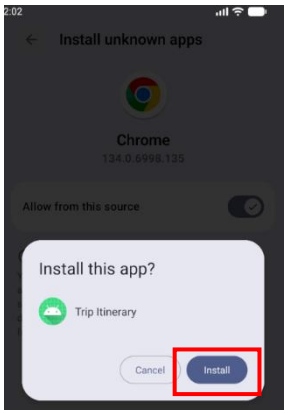
7. Then you will find the browser you used to download the APK file, and click on it.



8. Then you will turn on the “Allow from this source” setting.



9. Click “Install”. A Google Play scan might pop up, let it scan the app, and then you’re ready to find Trip Itinerary in your app drawer or on your home screen.

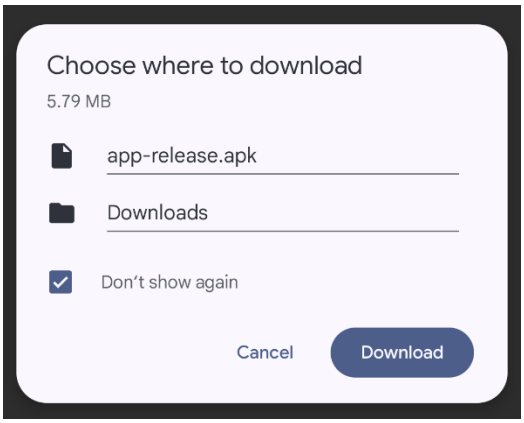


Installation Method 2: Using the APK file on PC through Android Studio.

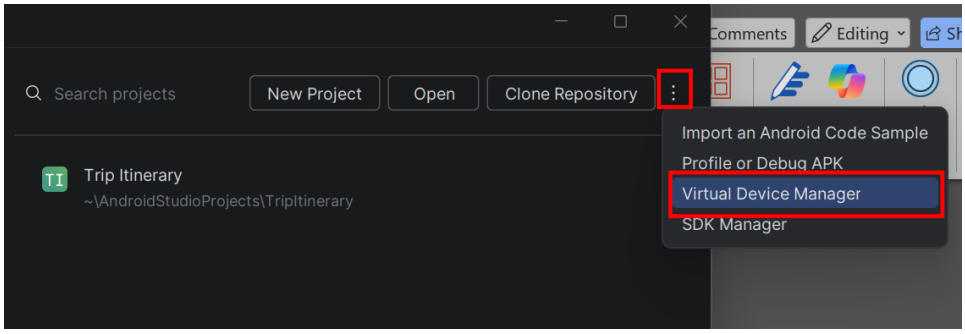
1. Download the APK file from the webpage.

   https://zoeyhankins.github.io/Trip_Itinerary_Website/

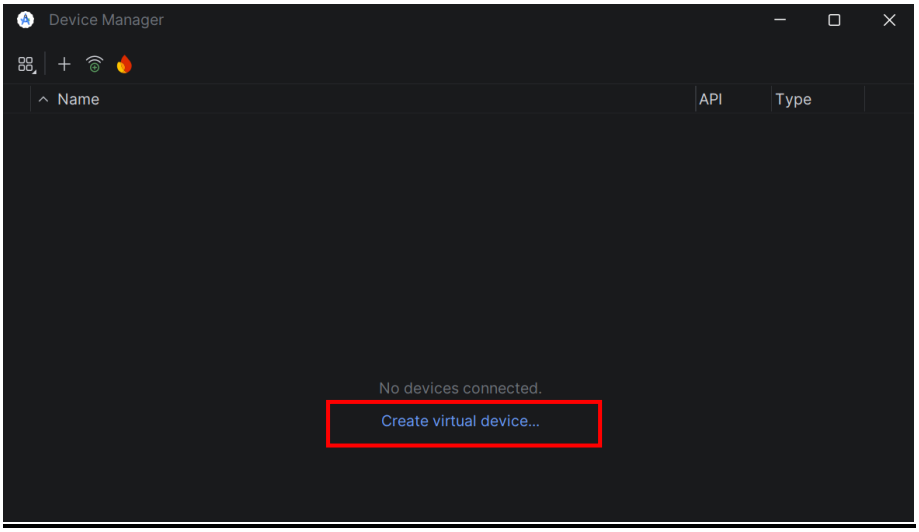
   The APK file will end up in a Downloads folder, or somewhere else if you specified.



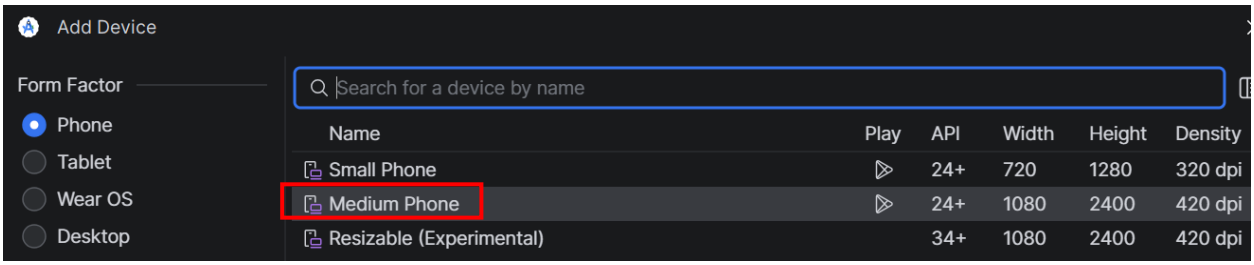
2. Open Android Studio, and to the right of "Clone Repository," there are three dots. Click on those, and then click "Virtual Device Manager".



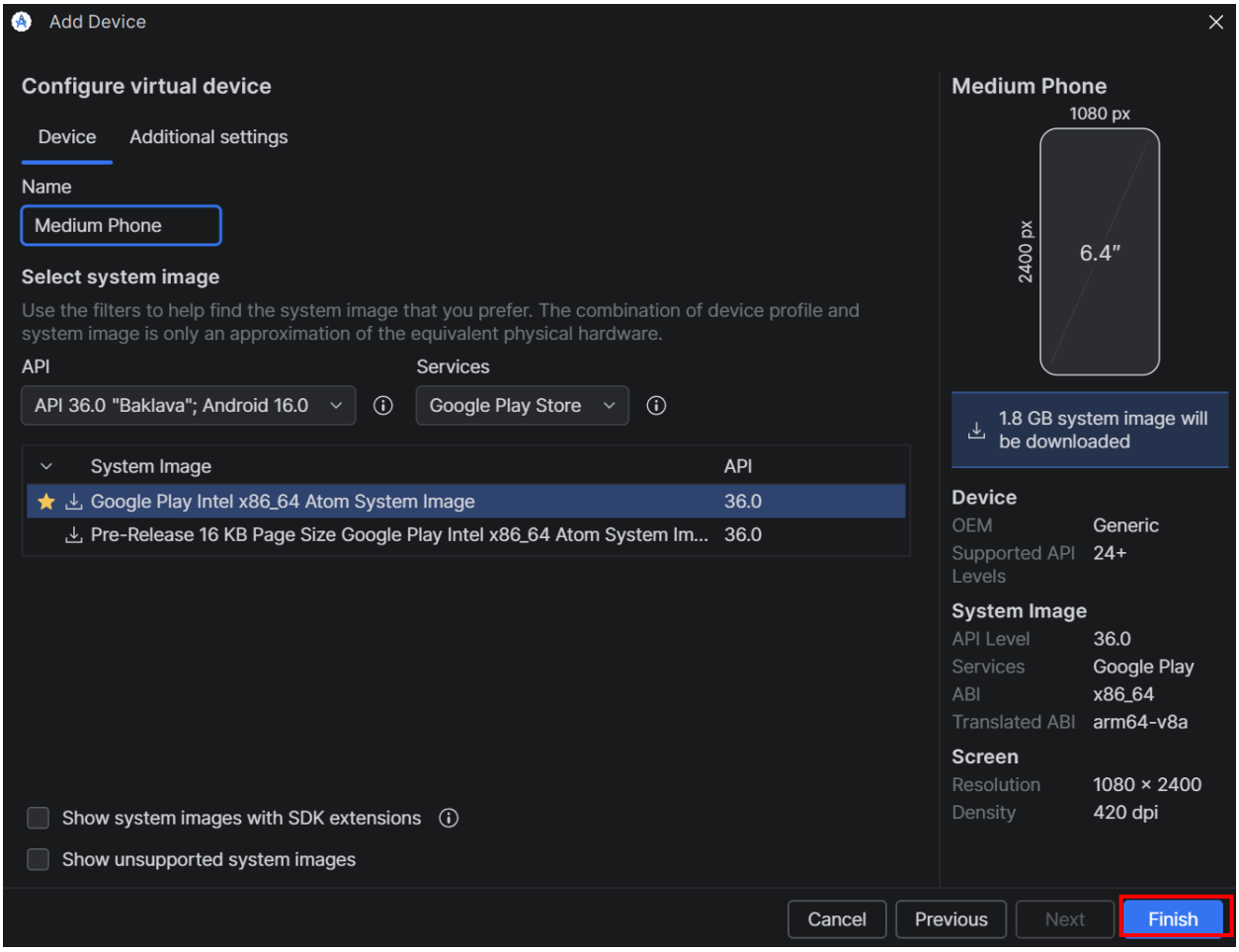
3. **If you already have a virtual device set up, skip to step 6.** If you don't have a virtual device set up, when you click on "Virtual Device Manager", it will be empty like this. You'll want to click "Create virtual device…".



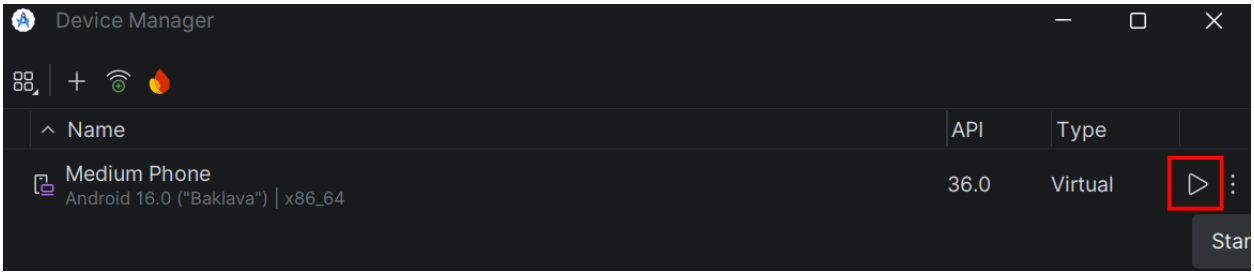
4. Once that opens, it will be a box with a ton of different devices. Medium phone will be the one you'll want to use.



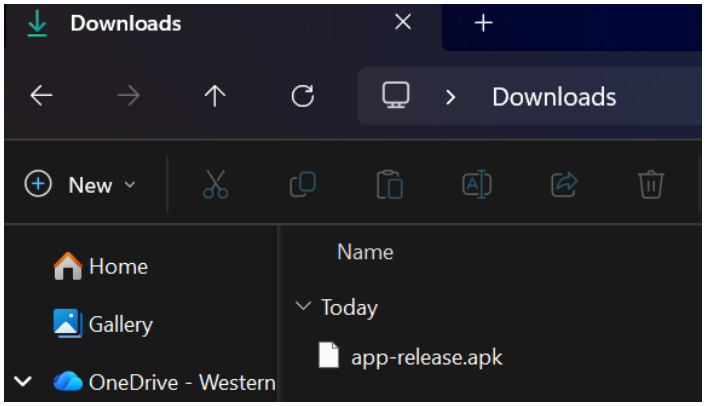
5. Then another screen will open with options. I would leave all settings the same. Unless you want to rename it, click finish.



6. Once the virtual device finishes setting up, this screen will be shown. Click the start arrow to start the device.

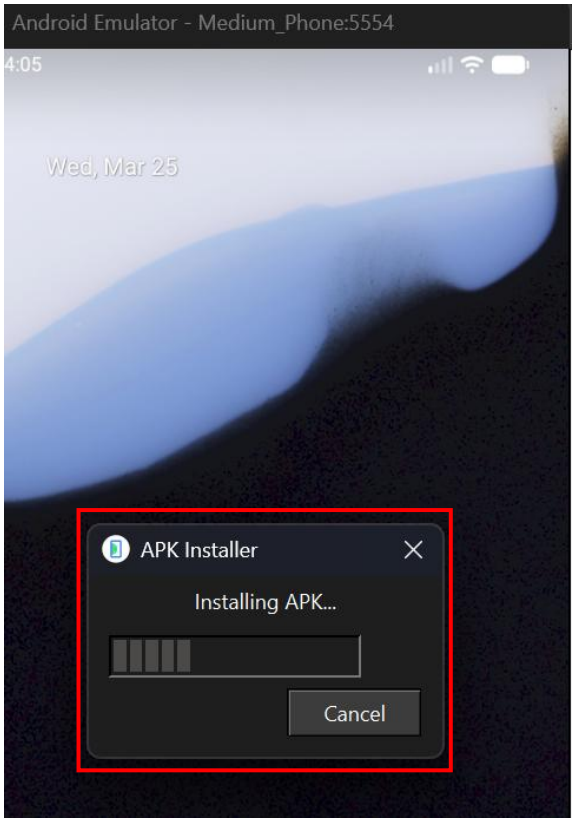


7. When the virtual device is finally open, you'll locate your APK file on your PC.

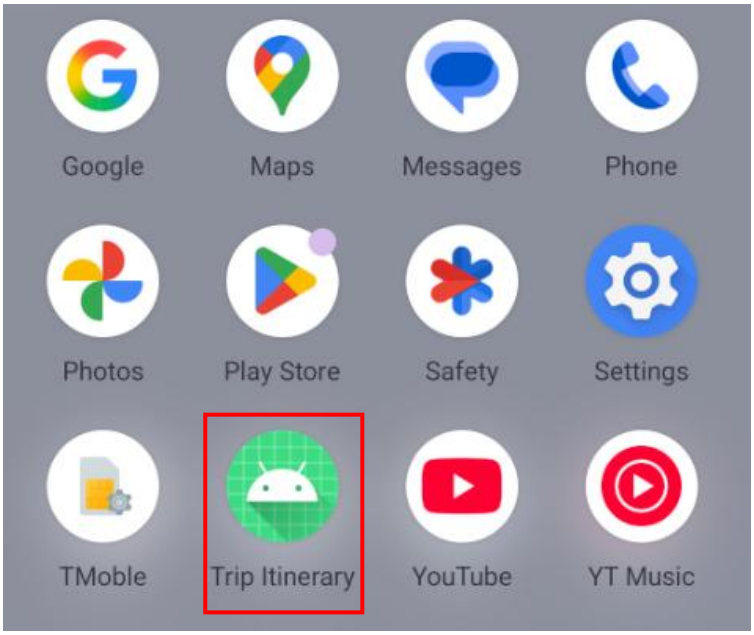


8. You will then drag and drop the “app-release.apk” from the downloads folder over to the virtual device.

9. The virtual device will then start installing the APK file and do the rest of the work for you.



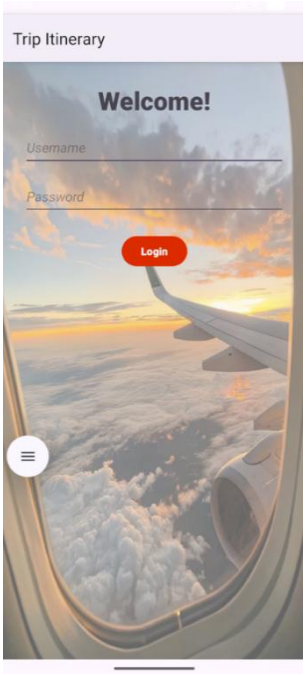
10. The last step is to navigate to your app drawer and open the application.



Using the Application

Login

1. Open the application, which will direct you to a Login Page.

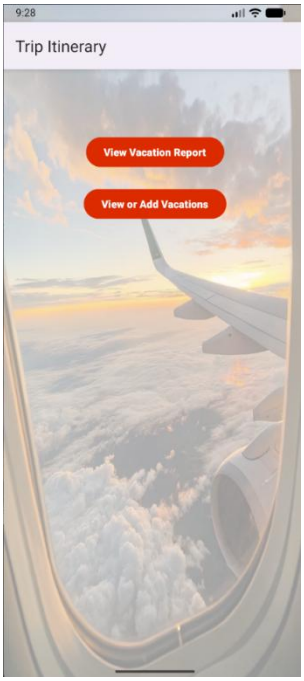


2. Enter the following credentials; the username does include a space and two capital letters. If you enter the username and password incorrectly, a message saying "invalid credentials" will be displayed at the bottom of the screen.

   Username: Trip User

   Password: password

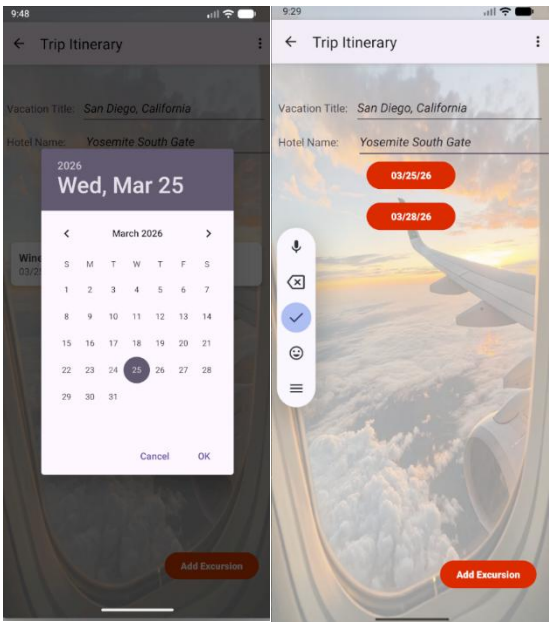
Once you're logged in, you will see the home screen.



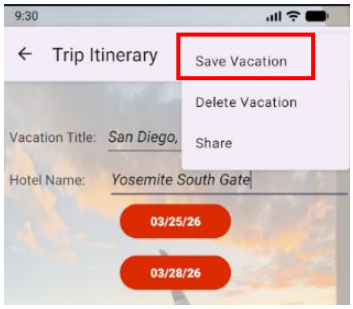
Adding and Editing Trips

1. To add your first trip, you will click on the “View or Add Vacations” button on the home page, which is shown above. You will click “Add Vacation”.

2. Fill out all the fields available. The dates have pop-up calendars when you click them.



3. When you're finished with the entry, in the upper right corner, there are three dots you click to save, delete, or share the vacation. Click "Save".

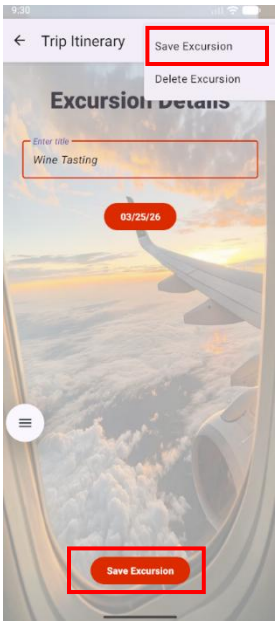


4. If you want to edit a vacation, you will click on the vacation you want to edit, change the text you input, and save the vacation again.

Adding and Editing Excursions

1. While inside a vacation entry, there is an “Add Excursion” button you will click.

2. Enter the excursion title and date.
3. You can save and delete the excursion from the upper-right menu. You can also use the save button at the bottom. Click “Save”.



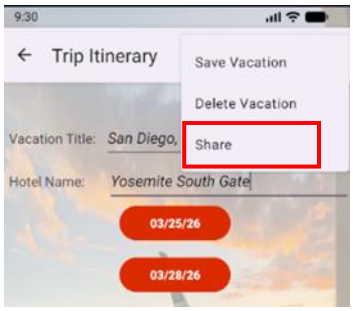
4. If you want to edit an excursion, you will click on the vacation, then click on the excursion you want to edit, and just change the text you input and save the excursion again.

View and Search Vacation Report

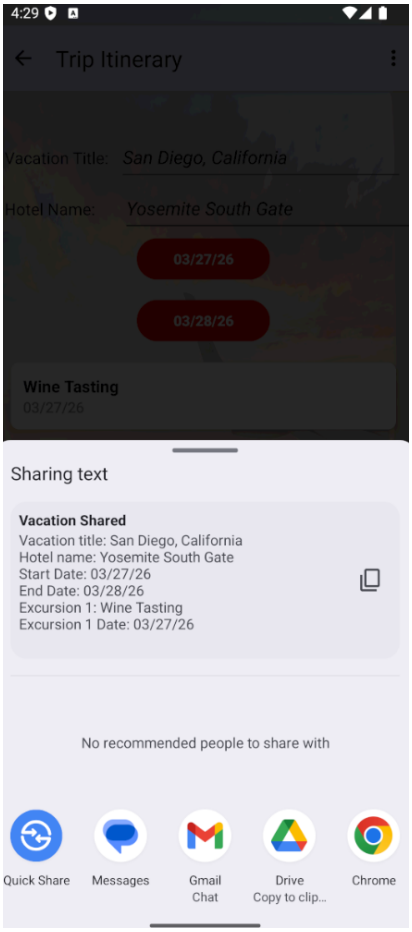
1. On the home page, you will click “View Vacation Report”.
2. On the report page, all vacations that are currently added are added to the report. You can then search for any vacation you have added.

Sharing Vacation Details

1. To share a vacation with or without excursions, you must go to the specific vacation and use the upper-right dots for the dropdown menu.

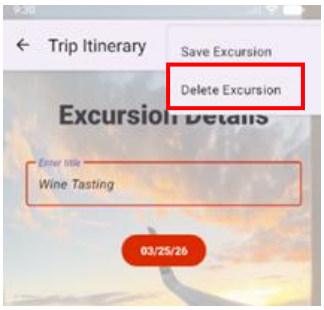
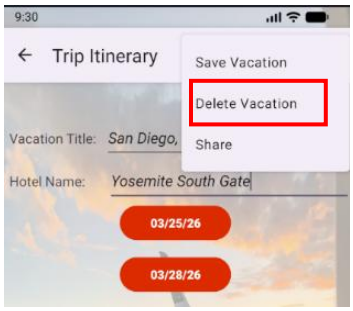


2. A menu at the bottom of the screen will then pop up and let you decide where to share.



Deleting a Vacation

1. Go to the specific vacation you want to delete.
2. Make sure you delete any associated excursions; you cannot delete an entry with excursions attached. You must click on the excursion and click “Delete Excursion” from the upper-right dropdown menu.



3. Once all excursions are deleted, you can delete the vacation using the upper-right dropdown menu buttons.